

commented-out code and dead code are never run by the computer, and might mislead programmers who later read the program's code. It's best to remove them and rely on a source control system like Git if you later need to add them back to your program.

Print debugging uses `print()` calls to display debugging information. Although this approach to debugging is easy, it's often faster in the long run to rely on a debugger and logs to diagnose bugs.

Variables with numeric suffixes, such as `x1`, `x2`, `x3`, and so on, are often best replaced with a single variable containing a list. Unlike in languages such as Java, in Python we use modules rather than classes to group functions together. A class that contains a single method or only static methods is a code smell suggesting that you should put that code into a module rather than a class. And although list comprehensions are a concise way to create list values, nested list comprehensions are usually unreadable.

Additionally, any exceptions handled with empty `except` blocks are a code smell that you're simply silencing the error rather than handling it. A short, cryptic error message is just as useless to the user as no error message.

Along with these code smells are the code smell myths: programming advice that is no longer valid or has, over time, proven counterproductive. These include putting only a single `return` statement or `try-except` block in each function, never using flag arguments or global variables, and believing that comments are unnecessary.

Of course, as with all programming advice, the code smells described in this chapter might or might not apply to your project or personal preferences. A best practice isn't an objective measure. As you gain more experience, you'll come to different conclusions about what code is readable or reliable, but the recommendations in this chapter outline issues to consider.